

ECM2433 Mock Exam 2 Mark Scheme

The C Family: C, C++, and Rust

This mark scheme follows the question numbering and marks from ECM2433_mock_exam_2.pdf. Equivalent correct answers should receive credit.

Question 1

(a) Question (4 marks): Dynamic C list allocation and out-of-bounds loop

Answer:

The loop condition is wrong:

```
for (int i = 0; i <= size; i++)
```

It should be:

```
for (int i = 0; i < size; i++)
```

`malloc(size * sizeof(int))` allocates space for indexes 0 to `size - 1`. The condition `i <= size` also writes `list[size]`, which is one past the allocated array.

Marking points:

- 1 mark: identifies that `<= size` should be `< size`.
- 1 mark: explains valid indexes are 0 to `size - 1`.
- 1 mark: says `list[size]` is out of bounds / heap buffer overflow.
- 1 mark: explains AddressSanitizer would report the invalid access with a diagnostic, often including file/line information, instead of the program silently continuing or crashing unpredictably.

(b) Question (4 marks): Difference between `sizeof` and `strlen`

Answer:

```
char hello[20] = "Hello world";
```

`sizeof hello` is the size of the whole array in bytes. Since `hello` is a `char[20]`, `sizeof hello` is 20.

`strlen(hello)` counts characters before the first null terminator. "Hello world" has 11 visible characters, so `strlen(hello)` is 11.

The array also contains a `'\0'` terminator after the `d`, and the remaining array elements are zero-initialised.

Marking points:

- 1 mark: `sizeof` gives the storage size of the array.
- 1 mark: `sizeof hello` is 20.
- 1 mark: `strlen` counts characters before `'\0'`.
- 1 mark: `strlen(hello)` is 11, not including the terminator or unused capacity.

(c) Question (4 marks): extern "C" and name mangling

Answer:

C++ compilers mangle names so overloaded functions can have distinct linker names. For example, `print(int)` and `print(double)` must become different symbols.

C does not use C++ name mangling. If a C++ file wants to call a function compiled from a `.c` file, the C++ declaration should use C linkage:

```
extern "C" {  
    void c_function(void);  
}
```

This tells the C++ compiler not to mangle the function name, so the linker can match it with the symbol produced by the C compiler.

Marking points:

- 1 mark: explains name mangling creates compiler/linker symbol names.
- 1 mark: links mangling to C++ overloading.
- 1 mark: says C does not use C++ name mangling.
- 1 mark: explains `extern "C"` gives C linkage so C++ can link to C functions.

(d) Question (8 marks): Generic `void * swap` function

Answer:

One correct implementation is:

```
#include <stddef.h>  
#include <stdlib.h>  
#include <string.h>  
  
void swap_anything(void *p, void *q, size_t size) {  
    unsigned char *tmp = malloc(size);  
  
    if (tmp == NULL) {  
        return;  
    }  
  
    memcpy(tmp, p, size);  
    memcpy(p, q, size);  
    memcpy(q, tmp, size);  
  
    free(tmp);  
}
```

This works because the function treats both objects as raw bytes. It copies `size` bytes from `p` to temporary storage, copies `q` into `p`, then copies the temporary bytes into `q`.

Marking points:

- 1 mark: uses `void *` parameters and `size_t size`.
- 1 mark: allocates or declares temporary storage of exactly `size` bytes.
- 1 mark: checks `malloc` failure if dynamic allocation is used.
- 1 mark: copies `p` into temporary storage.
- 1 mark: copies `q` into `p`.
- 1 mark: copies the temporary bytes into `q`.
- 1 mark: uses byte-wise copying such as `memcpy`/`memmove` or an `unsigned char *` loop.
- 1 mark: frees temporary storage if allocated dynamically.

(e) Question (5 marks): Unsafe `scanf("%s", buffer)`

Answer:

`scanf("%s", buffer)` is dangerous because `%s` reads a word of any length until whitespace. If the input is longer than the buffer, `scanf` writes past the end of the array, causing a buffer overflow.

If the buffer is:

```
char buffer[20];
```

a safer width-limited format is:

```
scanf("%19s", buffer);
```

The width is 19, not 20, because one byte is needed for the final `'\0'` terminator.

Marking points:

- 1 mark: explains `%s` stops at whitespace, not at the end of the buffer.
- 1 mark: identifies buffer overflow as the risk.
- 1 mark: gives a width-limited format string.
- 1 mark: leaves room for the null terminator.
- 1 mark: accepts `fgets(buffer, sizeof buffer, stdin)` as a better line-reading alternative.

Question 2

(a) Question (6 marks): Virtual destructors and binding in a C++ hierarchy

Answer:

The class uses a base pointer:

```
Sensor *s = new TempSensor("T1", 25.0);  
delete s;
```

If the base destructor is not virtual, deleting through `Sensor *` may only run the base destructor, so the derived object may not be cleaned up correctly. In the given code, `virtual ~Sensor()` fixes this because it makes destruction polymorphic.

A suitable base class is:

```
class Sensor {  
public:  
    std::string name;  
  
    Sensor(std::string n) : name(n) {}  
    virtual double read() { return 0.0; }  
    virtual ~Sensor() { std::cout << "Destructing Sensor"; }  
};
```

With a virtual destructor, deleting a `TempSensor` through a `Sensor *` runs the derived destructor first, then the base destructor.

Static binding means the function call is chosen from the static type known at compile time. Dynamic binding means the call is chosen at runtime using the real object type. Here, `read()` is virtual, so `s->read()` calls `TempSensor::read()` even though `s` has type `Sensor *`.

Marking points:

- 1 mark: identifies the danger of deleting a derived object through a base pointer without a virtual destructor.

- 1 mark: states the fix is a virtual base destructor.
- 1 mark: gives/describes correct destructor order: derived destructor then base destructor.
- 1 mark: defines static binding as compile-time selection from static type.
- 1 mark: defines dynamic binding as runtime virtual dispatch from actual object type.
- 1 mark: applies dynamic binding to `read()/TempSensor::read()`.

(b) Question (4 marks): Function overloading and name mangling

Answer:

C++ allows functions with the same name but different parameter types:

```
void log(int value);
void log(double value);
```

The linker needs unique names, so the compiler encodes extra information into the symbol names, such as the function name and parameter types. This is name mangling.

Conceptually:

```
log(int)    -> unique symbol for int version
log(double) -> unique symbol for double version
```

Marking points:

- 1 mark: says overloading allows same function name with different parameter lists.
- 1 mark: explains linker symbols must still be unique.
- 1 mark: explains mangling encodes parameter/type information.
- 1 mark: notes the exact mangled names are compiler-specific.

(c) Question (4 marks): Reading a full line after `std::cin >>`

Answer:

`std::cin >>` course reads only one whitespace-separated word. For input:

```
21 Computer Science
```

age becomes 21, but course only becomes "Computer". The word "Science" remains unread.

Use `std::getline` to read a full line:

```
int age;
std::string course;

std::cin >> age;
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::getline(std::cin, course);
```

The ignore removes the leftover newline after reading age.

Marking points:

- 1 mark: explains operator `>>` stops at whitespace.
- 1 mark: identifies that course becomes only "Computer".
- 1 mark: suggests `std::getline`.
- 1 mark: mentions handling the leftover newline after age.

(d) Question (6 marks): Sort `std::vector<Student>` by mark and name

Answer:

Assuming:

```
struct Student {
    std::string name;
    int mark;
};
```

A correct sort is:

```
#include <algorithm>
#include <string>
#include <vector>

std::sort(students.begin(), students.end(),
    [](const Student& a, const Student& b) {
        if (a.mark != b.mark) {
            return a.mark > b.mark;
        }
        return a.name < b.name;
    });
```

The comparator returns true when a should come before b. Higher marks come first, and equal marks are sorted alphabetically by name.

Marking points:

- 1 mark: uses `std::sort` with `begin()` and `end()`.
- 1 mark: uses a lambda comparator with two `Student` parameters.
- 1 mark: passes students by reference, preferably `const Student&`.
- 1 mark: sorts marks descending with `a.mark > b.mark`.
- 1 mark: handles equal marks as a separate case.
- 1 mark: sorts equal-mark names ascending with `a.name < b.name`.

(e) Question (5 marks): Template `Pair<T>` with equality check

Answer:

One simple answer is:

```
template<typename T>
class Pair {
    T first;
    T second;

public:
    Pair(T first, T second) : first(first), second(second) {}

    bool are_equal() const {
        return first == second;
    }
};
```

`are_equal()` uses `operator==` on `T`. This works for types where equality is defined, including user-defined types that overload `operator==`.

Marking points:

- 1 mark: declares `template<typename T>` or equivalent.

- 1 mark: stores two values of type T.
- 1 mark: provides a constructor or other way to initialise both values.
- 1 mark: implements `are_equal()` returning `bool`.
- 1 mark: uses operator `==` and notes T must support equality comparison.

Question 3

(a) Question (4 marks): `Option<T>` *return and match*

Answer:

A suitable signature is:

```
fn find_task(tasks: &[String], keyword: &str) -> Option<usize>
```

One implementation:

```
fn find_task(tasks: &[String], keyword: &str) -> Option<usize> {
    tasks.iter().position(|task| task.contains(keyword))
}
```

Handling it with match:

```
match find_task(&tasks, "urgent") {
    Some(index) => println!("found at {index}"),
    None => println!("not found"),
}
```

Marking points:

- 1 mark: takes a borrowed slice such as `&[String]`.
- 1 mark: takes the keyword as a borrowed string such as `&str`.
- 1 mark: returns `Option<usize>`.
- 1 mark: handles both `Some(index)` and `None` using match.

(b) Question (4 marks): *Interior mutability with* `Arc<T>` *and* `Mutex<T>`

Answer:

`Arc<T>` gives shared ownership across threads using atomic reference counting. It lets multiple threads own the same allocation safely.

`Mutex<T>` provides interior mutability. Even if the `Mutex` itself is shared, code can call `lock()` to get controlled mutable access to the inner T.

Together:

```
use std::sync::{Arc, Mutex};

let shared = Arc::new(Mutex::new(0));
```

`Arc` shares the mutex between threads. `Mutex` ensures only one thread can mutate the value at a time.

Marking points:

- 1 mark: `Arc` gives thread-safe shared ownership.
- 1 mark: `Mutex` protects mutable access with locking.

- 1 mark: explains interior mutability means mutation through a shared wrapper under runtime rules.
- 1 mark: says `Arc<Mutex<T>>` allows shared mutable state across threads with exclusive access while locked.

(c) Question (6 marks): Rust borrow rules with vector push and existing reference

Answer:

The code fails because `r1` is an immutable reference into `tasks`:

```
let r1 = &tasks[0];
```

Then this line needs a mutable borrow of `tasks`:

```
tasks.push(String::from("Task 2"));
```

Rust does not allow a mutable borrow while an immutable borrow that will be used later is still active.

There is also a practical reason: pushing into a `Vec` may reallocate its buffer. If reallocation happens, references to old elements could become invalid. Rust prevents that at compile time.

One fix is to use `r1` before the mutation:

```
let r1 = &tasks[0];
println!("{r1}");
tasks.push(String::from("Task 2"));
```

Another fix is to clone the value:

```
let r1 = tasks[0].clone();
tasks.push(String::from("Task 2"));
println!("{r1}");
```

Marking points:

- 1 mark: identifies `r1` as an immutable borrow from `tasks`.
- 1 mark: identifies `push` as requiring mutable access to `tasks`.
- 1 mark: states Rust forbids mutable and active immutable borrows at the same time.
- 1 mark: mentions `r1` is used after the push, so its borrow is still live.
- 1 mark: explains vector push can reallocate and invalidate references.
- 1 mark: gives a valid fix such as print before push or clone the string.

(d) Question (4 marks): Iterator pipeline filtering and doubling

Answer:

Using `.iter()`:

```
let result: Vec<i32> = values
    .iter()
    .filter(|n| **n > 10)
    .map(|n| *n * 2)
    .collect();
```

Using `.into_iter()`:

```
let result: Vec<i32> = values
    .into_iter()
    .filter(|n| *n > 10)
    .map(|n| n * 2)
    .collect();
```

Both keep values greater than 10, double them, and collect into a new vector.

Marking points:

- 1 mark: uses an iterator method chain.
- 1 mark: filters values greater than 10.
- 1 mark: maps each kept value to double it.
- 1 mark: collects into a Vec, with correct reference handling.

(e) Question (4 marks): Deref coercion with Box<Song>

Answer:

Box<Song> owns a Song stored on the heap. Box<T> implements Deref<Target = T>, so Rust can automatically convert a reference to a Box<Song> into a reference to a Song when needed.

Example:

```
struct Song {
    title: String,
}

fn play(song: &Song) {
    println!("{}", song.title);
}

let boxed = Box::new(Song {
    title: String::from("Track"),
});

play(&boxed);
```

&boxed has type &Box<Song>, but deref coercion lets Rust treat it as &Song for the function call.

Marking points:

- 1 mark: says Box<Song> owns a heap-allocated Song.
- 1 mark: says Box<T> dereferences to T.
- 1 mark: explains &Box<Song> can be coerced to &Song.
- 1 mark: distinguishes borrowing from moving ownership of the Song.

Question 4

(a) Question (6 marks): Error handling in C and Rust

Answer:

C often uses return codes:

```
int read_count(void);
```

The problem is ambiguity. If 0, -1, or another integer could be both a valid result and an error code, the caller needs extra rules or an output parameter to tell success from failure.

For example, `atoi("abc")` and `atoi("0")` both produce 0, so the return value alone cannot reliably distinguish invalid input from valid zero.

Rust usually uses `Result<T, E>` for recoverable errors:


```
fn read_count(text: &str) -> Result<i32, std::num::ParseIntError> {
    let n = text.parse::<i32>()?;
    Ok(n)
}
```

The `?` operator unwraps the `Ok` value or returns the `Err` early from the current function. This avoids repeated manual `match` or `if let` checks while still preserving the error.

Marking points:

- 1 mark: describes C return-code style.
- 1 mark: explains the ambiguity when an error code can also be a valid integer.
- 1 mark: gives a relevant example such as 0 from valid zero versus invalid input.
- 1 mark: identifies Rust `Result<T, E>` as carrying success or error explicitly.
- 1 mark: explains `?` returns early on `Err`.
- 1 mark: explains `?` reduces manual checking while preserving the error value.

(b) Question (6 marks): C++ smart pointers and ownership

Answer:

Smart pointers are RAII objects that manage dynamically allocated objects automatically. When the smart pointer is destroyed, it releases the owned resource according to its ownership rules.

`std::unique_ptr<T>` has unique ownership. Only one `unique_ptr` owns the object at a time. It cannot be copied, but it can be moved.

```
std::unique_ptr<Sensor> s = std::make_unique<TempSensor>("T1", 25.0);
```

`std::shared_ptr<T>` has shared ownership. Several `shared_ptr`s can own the same object. The object is destroyed when the last shared owner is gone.

```
auto a = std::make_shared<int>(5);
auto b = a; // both share ownership
```

Marking points:

- 1 mark: defines smart pointers as objects that manage pointer/resource lifetime.
- 1 mark: connects smart pointers to RAII / automatic cleanup.
- 1 mark: explains `unique_ptr` has one owner.
- 1 mark: explains `unique_ptr` can be moved but not copied.
- 1 mark: explains `shared_ptr` uses shared/reference-counted ownership.
- 1 mark: says the shared object is destroyed when the last `shared_ptr` is destroyed or reset.

(c) Question (8 marks): Rust thread move closures and mpsc channels

Answer:

`std::thread::spawn` may run the new thread after the spawning function has continued or returned. Because of that, the closure usually needs to own the values it uses. The `move` keyword moves captured values into the closure.

Example:

```
use std::thread;

let text = String::from("work");

let handle = thread::spawn(move || {
```

```
println!("{text}");
});

handle.join().unwrap();
```

Without move, the thread might try to borrow stack data from the parent thread for too long.

An mpsc channel lets worker threads send results back to one receiver:

```
use std::sync::mpsc;
use std::thread;

let (tx, rx) = mpsc::channel();
let mut handles = Vec::new();

for id in 0..4 {
    let tx = tx.clone();

    handles.push(thread::spawn(move || {
        tx.send(id * 10).unwrap();
    }));
}

drop(tx);

for value in rx {
    println!("{value}");
}

for handle in handles {
    handle.join().unwrap();
}
```

Each worker gets its own cloned sender. All sender clones still send to the same receiver. `drop(tx)` removes the original sender so the receiver loop can finish once all worker senders are gone.

Marking points:

- 1 mark: explains spawned threads may outlive the current stack frame.
- 1 mark: explains move transfers captured values into the closure.
- 1 mark: notes this prevents borrowing parent stack data for too long.
- 1 mark: identifies mpsc as multiple producer, single consumer.
- 1 mark: explains cloning tx gives each worker a sender to the same channel.
- 1 mark: shows or describes send from each worker.
- 1 mark: explains rx receives/iterates over worker results.
- 1 mark: explains dropping the original sender and/or joining handles to allow clean completion.

(d) Question (5 marks): C++ operator+ *prototype for* MyTime

Answer:

As a member operator:

```
class MyTime {
public:
    MyTime operator+(int seconds) const;
};
```

Out-of-class definition shape:

```
MyTime MyTime::operator+(int seconds) const {
    MyTime result = *this;
    /* add seconds to result */
    return result;
}
```

The return type is `MyTime` because adding seconds should produce a new time value. The parameter is `int seconds`. The trailing `const` means the operation does not modify the left-hand operand.

A non-member version is also acceptable:

```
MyTime operator+(const MyTime& time, int seconds);
```

Marking points:

- 1 mark: uses the correct operator name `operator+`.
- 1 mark: returns `MyTime`.
- 1 mark: takes an `int seconds` parameter.
- 1 mark: gives either a valid member or non-member prototype.
- 1 mark: includes `const` where appropriate or explains the operation should not mutate the original object.